



Longest Substring with Same Letters after Replacement (hard)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time Complexity
 - Space Complexity

Problem Statement

Given a string with lowercase letters only, if you are allowed to **replace no more than k letters** with any letter, find the **length of the longest substring having the same letters** after replacement.

Example 1:

Input: String="aabccbb", k=2

Output: 5

Explanation: Replace the two 'c' with 'b' to have the longest repeating substring "bbbb".

Example 2:

Input: String="abbc", k=1

Output: 4

Explanation: Replace the 'c' with 'b' to have the longest repeating substring "bbbb".

Example 3:

Input: String="abccde", k=1

Output: 3

Explanation: Replace the 'b' or 'd' with 'c' to have the longest repeating substring "ccc".

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1
2 class CharacterReplacement {
3     public static int findLength(String str, int k) {
4         // TODO: Write your code here
```



```

5    int s=0, max=0, repeatCount=0;
6    Map<Character, Integer> hm = new HashMap<>();
7    for(int e=0; e< str.length();e++){
8        char ch = str.charAt(e);
9        hm.put(ch, hm.getOrDefault(ch, 0) +1);
10       repeatCount = Math.max(repeatCount, hm.get(ch));
11       if(e-s+1-repeatCount > k){
12           ch = str.charAt(s);
13           hm.put(ch, hm.get(ch) - 1);
14           s++;
15       }
16
17       max = Math.max(max, e-s+1);
18   }return max;
19 }
20 }
21

```

Test

Save

Reset



Solution

This problem follows the **Sliding Window** pattern, and we can use a similar dynamic sliding window strategy as discussed in [Longest Substring with Distinct Characters](#). We can use a HashMap to count the frequency of each letter.

- We will iterate through the string to add one letter at a time in the window.
- We will also keep track of the count of the maximum repeating letter in **any** window (let's call it `maxRepeatLetterCount`).
- So, at any time, we know that we do have a window with one letter repeating `maxRepeatLetterCount` times; this means we should try to replace the remaining letters.
 - If the remaining letters are less than or equal to `k`, we can replace them all.
 - If we have more than `k` remaining letters, we should shrink the window as we cannot replace more than `k` letters.

While shrinking the window, we don't need to update `maxRepeatLetterCount`. Since we are only interested in the longest valid substring, our sliding windows do not have to shrink, even if a window may cover an invalid substring. Either we expand the window by appending a character to the right or we shift the entire window to the right by one. We only expand the window when the frequency of the newly added character exceeds the historical maximum frequency (from a previous window that included a valid substring). In other words, we do not need to know the exact maximum count of the current window. The only thing we need to know is whether the maximum count exceeds the historical maximum count, and that can only happen because of the newly added char.

Code

Here is what our algorithm will look like:

```

1  import java.util.*;
2
3  class CharacterReplacement {

```



```

4 public static int findLength(String str, int k) {
5     int windowStart = 0, maxLength = 0, maxRepeatLetterCount = 0;
6     Map<Character, Integer> letterFrequencyMap = new HashMap<>();
7     // try to extend the range [windowStart, windowEnd]
8     for (int windowEnd = 0; windowEnd < str.length(); windowEnd++) {
9         char rightChar = str.charAt(windowEnd);
10        letterFrequencyMap.put(rightChar, letterFrequencyMap.getOrDefault(rightChar, 0) + 1);
11
12        // we don't need to place the maxRepeatLetterCount under the below 'if', see the
13        // explanation in the 'Solution' section above.
14        maxRepeatLetterCount = Math.max(maxRepeatLetterCount, letterFrequencyMap.get(rightChar));
15
16        // current window size is from windowStart to windowEnd, overall we have a letter which is
17        // repeating 'maxRepeatLetterCount' times, this means we can have a window which has one letter
18        // repeating 'maxRepeatLetterCount' times and the remaining letters we should replace.
19        // if the remaining letters are more than 'k', it is the time to shrink the window as we
20        // are not allowed to replace more than 'k' letters
21        if (windowEnd - windowStart + 1 - maxRepeatLetterCount > k) {
22            char leftChar = str.charAt(windowStart);
23            letterFrequencyMap.put(leftChar, letterFrequencyMap.get(leftChar) - 1);
24            windowStart++;
25        }
26
27        maxLength = Math.max(maxLength, windowEnd - windowStart + 1);
28    }
29
30    return maxLength;
31 }
32
33 public static void main(String[] args) {
34     System.out.println(CharacterReplacement.findLength("aabccbb", 2));
35     // System.out.println(CharacterReplacement.findLength("abbcb", 1));
36     // System.out.println(CharacterReplacement.findLength("abccde", 1));
37 }
38 }
39

```

Run

Save

Reset



Time Complexity

The above algorithm's time complexity will be $O(N)$, where 'N' is the number of letters in the input string.

Space Complexity

As we expect only the lower case letters in the input string, we can conclude that the space complexity will be $O(26)$ to store each letter's frequency in the **HashMap**, which is asymptotically equal to $O(1)$.

[< Back](#)
[✔ Completed](#)
[Next >](#)

Longest Substring with Distinct Characters (hard)

Longest Subarray with Ones after Replacement (hard)